Звіт з практичної роботи №11

Обробка винятків: програми, які не падають на поганих файлах

Студент:	Nikita Krilovets
Група:	IT-31
День народження (day):	8
Профіль:	data/profile.txt (з роботи 22)
Витрати:	data/expenses.csv (з роботи 22)

Мета роботи

Навчитися перехоплювати винятки, що виникають під час роботи з файлами, обирати правильний тип і порядок гілок **except**, застосовувати **else** і **finally**, обробляти виняток на тому рівні програми, де є достатньо контексту, та завершувати програму зрозумілим повідомленням і кодом завершення замість traceback.

Усі програми першим рядком виводять ім'я, прізвище та групу (*Nikita Krilovets, IT-31*), повідомлення про помилки виводять у **sys.stderr**, а результат — у звичайний вивід. Код завершення після кожного запуску переглядається змінною **$LASTEXITCODE** (аналог **echo $?** у PowerShell).

Підготовка

Створено каталог проєкту **practice24** з трьома файлами завдань (**task1.py**, **task2.py**, **task3.py**). Командою **mkdir** створено каталог **data**, а командою **New-Item** — порожній файл **data/expenses.csv**, який використовується в завданнях 1 і 3.

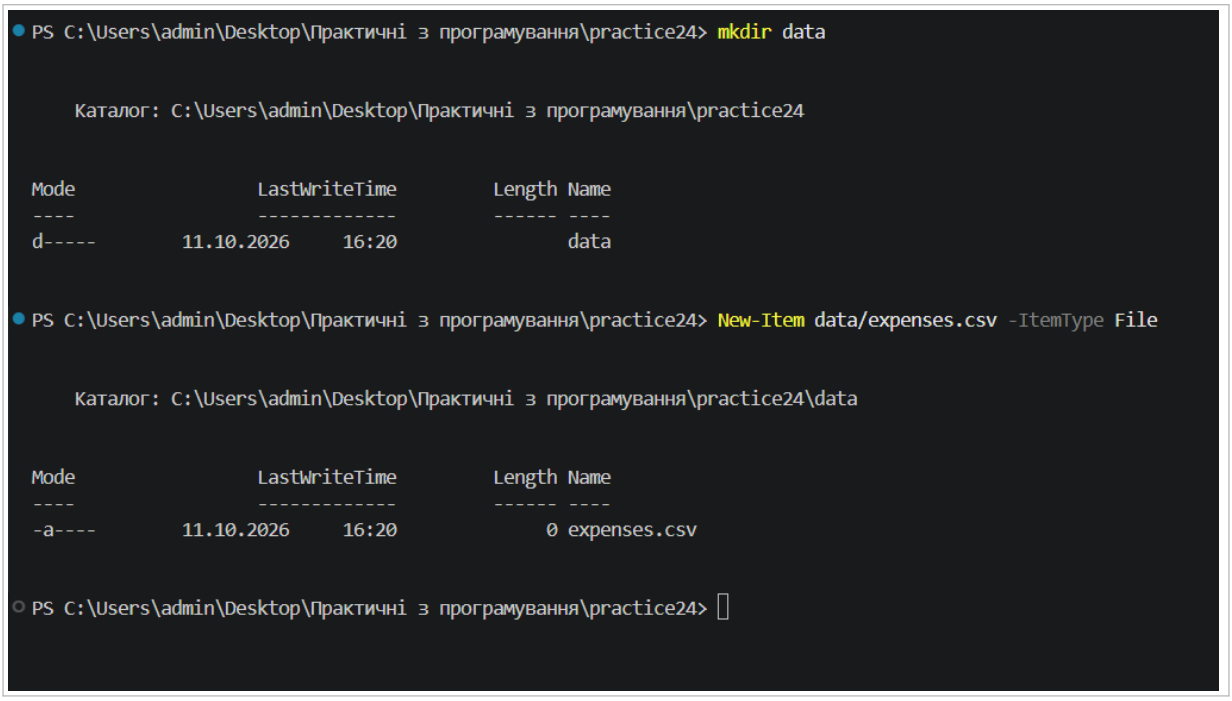


Рис. 1 — Підготовка: створення каталогу data і файлу data/expenses.csv

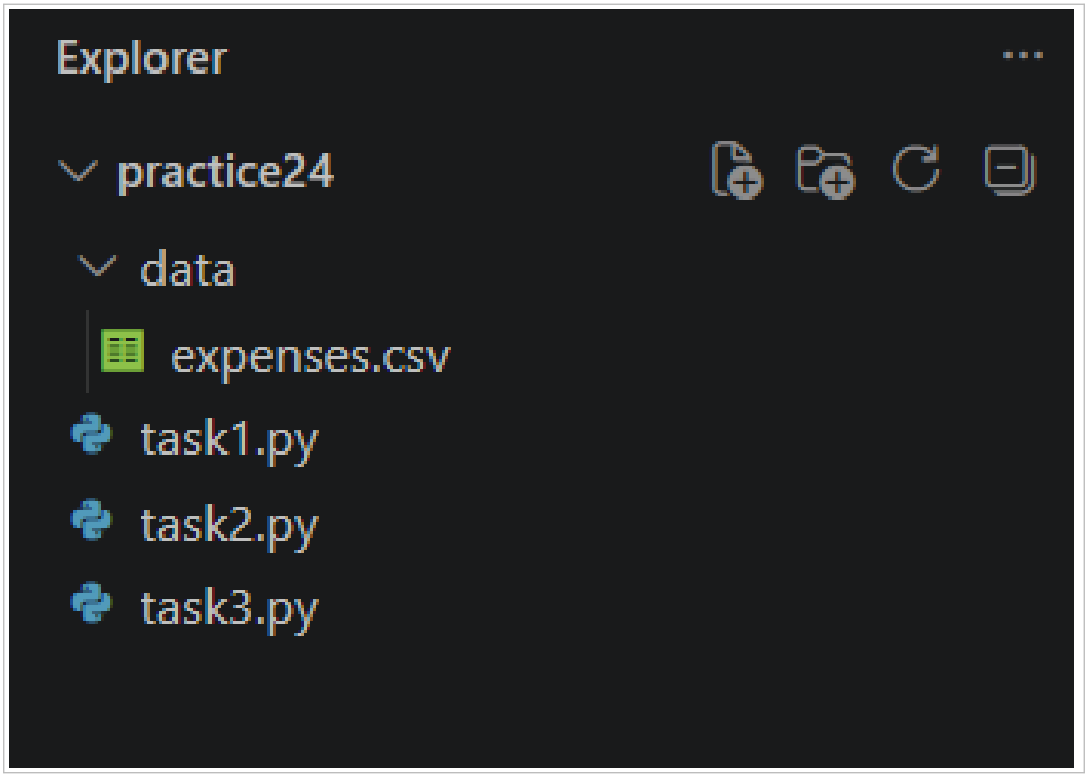


Рис. 2 — Підготовка: структура проєкту practice24 в Explorer

Завдання 1. Чому файл не читається (task1.py)

Утиліта отримує з командного рядка один або кілька шляхів. Для кожного файлу вона показує кількість рядків і перший рядок або пояснює причину помилки разом з назвою типу винятку. Порожній файл вважається помилкою.

Код програми

Функція **read_file(path)** відкриває файл у режимі читання з кодуванням UTF-8 і зчитує рядки. Якщо список рядків порожній, явно викликається **ValueError("Файл порожній")**. В одній гілці **except (OSError, UnicodeError, ValueError)** перехоплюються всі очікувані проблеми: немає файлу, каталог замість файлу, немає прав, файл не в UTF-8 та порожній файл. Повідомлення з назвою типу винятку (**type(error).__name__**) виводиться у **sys.stderr**, функція повертає **False**.

У функції **main()** шляхи беруться з **sys.argv[1:]**. Якщо шляхів немає — виводиться підказка і повертається код **2**. Якщо хоча б один файл не прочитано — код **1**, інакше — **0**. Код завершення передається через **sys.exit(main())**.

task1.py

```python

import sys
from pathlib import Path

print("Nikita Krilovets, IT-31")


def read_file(path):
    try:
        with path.open("r", encoding="utf-8") as file:
            lines = file.readlines()

        if not lines:
            raise ValueError("Файл порожній")

        print(f"{path}: рядків – {len(lines)}")
        print(f"Перший рядок: {lines[0].rstrip()}")

        return True

    except (OSError, UnicodeError, ValueError) as error:
        print(
            f"Помилка для {path}: {type(error).__name__}: {error}",
            file=sys.stderr
        )
        return False


def main():
    paths = sys.argv[1:]

    if not paths:
        print("Використання: python task1.py <шлях> [<шлях> ...]",
              file=sys.stderr)
        return 2

    success = True

    for item in paths:
        if not read_file(Path(item)):
            success = False

    return 0 if success else 1


if __name__ == "__main__":
    sys.exit(main())

```

Рис. 3 — Завдання 1: код програми task1.py

Результати перевірки

- **data/expenses.csv** — файл прочитано: 4 рядки, перший рядок — заголовок; код завершення 0.
- **data/missing_Krilovets.txt** — неіснуючий файл із прізвищем у назві: **FileNotFoundError**; код 1.
- **data** — каталог замість файлу: у Windows спроба відкрити каталог як файл дає **PermissionError** (у Linux це був би **IsADirectoryError**); код 1.
- **data/empty.txt** — порожній файл, створений командою **New-Item**: **ValueError: Файл порожній**; код 1.
- Запуск без аргументів — підказка про використання; код 2.

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task1.py data/expenses.csv
Nikita Krilovets, IT-31
data\expenses.csv: рядків — 4
Перший рядок: date;category;amount;note
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
0
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task1.py data/missing_Krilovets.txt
Nikita Krilovets, IT-31
Помилка для data\missing_Krilovets.txt: FileNotFoundError: [Errno 2] No such file or directory: 'data\\missing_Krilovets.txt'
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task1.py data
Nikita Krilovets, IT-31
Помилка для data: PermissionError: [Errno 13] Permission denied: 'data'
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> New-Item -ItemType File -Force data/empty.txt


    Каталог: C:\Users\admin\Desktop\Практичні з програмування\practice24\data


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        11.10.2026     16:36              0 empty.txt


PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task1.py data/empty.txt
Nikita Krilovets, IT-31
Помилка для data\empty.txt: ValueError: Файл порожній
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task1.py
Nikita Krilovets, IT-31
Використання: python task1.py <шлях> [<шлях> ...]
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
2
```

Рис. 4 — Завдання 1: запуски програми та коди завершення

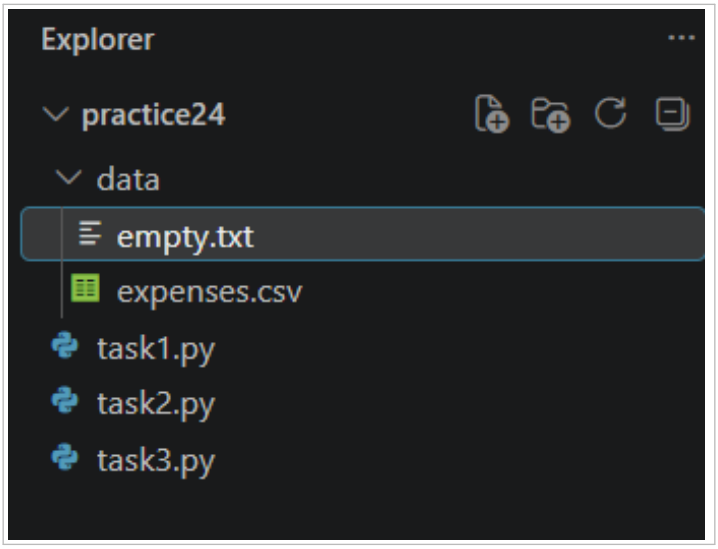


Рис. 5 — Завдання 1: каталог data після створення порожнього файлу empty.txt

Завдання 2. Копіювання файлів (task2.py)

Утиліта є спрощеним аналогом команди **cp**: **python task2.py [--force] <джерело> <призначення>**. Коди завершення: 0 — файл скопійовано, 1 — помилка копіювання, 2 — неправильний виклик.

Код програми

У функції **main()** з аргументів прибирається прапорець **--force**; якщо після цього аргументів не рівно два, виводиться підказка в **sys.stderr** і повертається код **2**. Шляхи перетворюються на об'єкти **Path**.

task2.py

```python

import sys
import shutil
from pathlib import Path

print("Nikita Krilovets, IT-31")


def main():
    args = sys.argv[1:]
    force = False

    if "--force" in args:
        force = True
        args.remove("--force")

    if len(args) != 2:
        print(
            "Використання: python task2.py [--force] <джерело> <призначення>",
            file=sys.stderr
        )
        return 2

    source = Path(args[0])
    destination = Path(args[1])

```

Рис. 6 — Завдання 2: код task2.py (початок — розбір аргументів)

Усередині блоку **try** послідовно перевіряються всі ситуації, і для кожної викликається відповідний виняток: джерела не існує (**FileNotFoundError**), джерело — каталог (**IsADirectoryError**), каталогу призначення не існує (**FileNotFoundError**), призначення — каталог (**IsADirectoryError**), джерело і призначення — той самий файл (**shutil.SameFileError**), файл призначення вже існує без **--force** (**FileExistsError**). Якщо всі перевірки пройдено, файл копіюється через **shutil.copy2** і повертається код 0.

Порядок гілок **except** має значення: спочатку йде вужчий **shutil.SameFileError** (повідомлення без назви типу), а після нього — загальна гілка **except (OSError, shutil.Error)** з назвою типу винятку. Якби порядок був зворотним, виняток **SameFileError** перехоплювався б загальною гілкою.

```python

    try:
        if not source.exists():
            raise FileNotFoundError(f"Джерело не знайдено: {source}")

        if source.is_dir():
            raise IsADirectoryError(f"Джерело є папкою: {source}")

        if not destination.parent.exists():
            raise FileNotFoundError(
                f"Папка призначення не існує: {destination.parent}"
            )

        if destination.is_dir():
            raise IsADirectoryError(
                f"Призначення є папкою: {destination}"
            )

        if source.resolve() == destination.resolve():
            raise shutil.SameFileError("Джерело та призначення — той самий файл")

        if destination.exists() and not force:
            raise FileExistsError(
                f"Файл уже існує: {destination}. Використай --force"
            )

        shutil.copy2(source, destination)
        print(f"Файл скопійовано: {source} -> {destination}")
        return 0

    except shutil.SameFileError as error:
        print(f"Помилка: {error}", file=sys.stderr)
        return 1
    except (OSError, shutil.Error) as error:
        print(f"Помилка копіювання: {type(error).__name__}: {error}",
              file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())

```

Рис. 7 — Завдання 2: код task2.py (перевірки, копіювання та обробка винятків)

Результати перевірки

Перед запуском створено тестовий файл **source.txt** командою **Set-Content**. Перевірено такі випадки:

- копіювання **source.txt → copy.txt** — успіх, код 0;
- повторне копіювання без **--force** — **FileExistsError**, код 1;
- копіювання з **--force** — файл перезаписано, код 0;
- неіснуюче джерело **missing.txt** — **FileNotFoundError**, код 1;
- неіснуюча папка призначення **missing/copy.txt** — **FileNotFoundError**, код 1;
- джерело і призначення — той самий файл — помилка, код 1;
- джерело — каталог **data** — **IsADirectoryError**, код 1;
- призначення — каталог **data** — **IsADirectoryError**, код 1;
- виклик без аргументів — підказка, код 2.

```
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> "Тестовий файл" | Set-Content source.txt
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py source.txt copy.txt
Nikita Krilovets, IT-31
Файл скопійовано: source.txt -> copy.txt
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
0
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py source.txt copy.txt
Nikita Krilovets, IT-31
Помилка копіювання: FileExistsError: Файл уже існує: copy.txt. Використай --force
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py --force source.txt copy.txt
Nikita Krilovets, IT-31
Файл скопійовано: source.txt -> copy.txt
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
0
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py missing.txt copy2.txt
Nikita Krilovets, IT-31
Помилка копіювання: FileNotFoundError: Джерело не знайдено: missing.txt
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py source.txt missing/copy.txt
Nikita Krilovets, IT-31
Помилка копіювання: FileNotFoundError: Папка призначення не існує: missing
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py source.txt source.txt
Nikita Krilovets, IT-31
Помилка: Джерело та призначення — той самий файл
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py data copy.txt
Nikita Krilovets, IT-31
Помилка копіювання: IsADirectoryError: Джерело є папкою: data
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py source.txt data
Nikita Krilovets, IT-31
Помилка копіювання: IsADirectoryError: Призначення є папкою: data
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
1
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> python task2.py
Nikita Krilovets, IT-31
Використання: python task2.py [--force] <джерело> <призначення>
PS C:\Users\admin\Desktop\Практичні з програмування\practice24> $LASTEXITCODE
2
```

Рис. 8 — Завдання 2: запуски програми та коди завершення

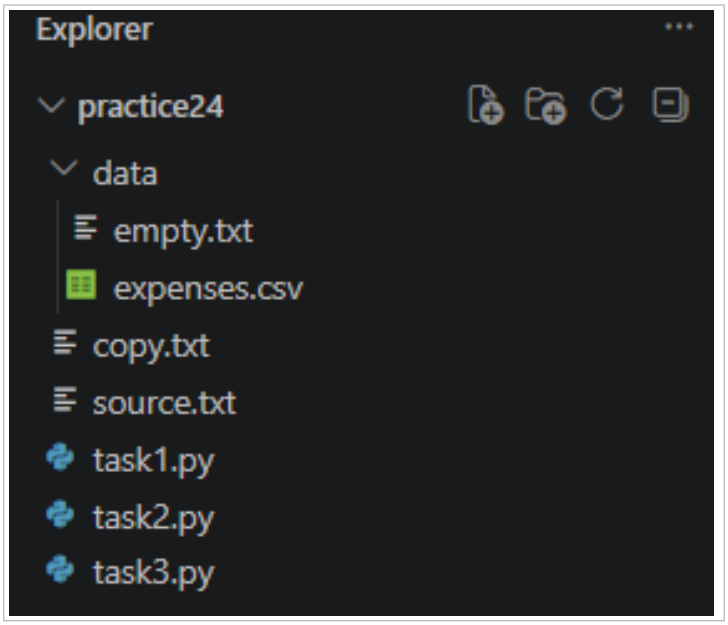


Рис. 9 — Завдання 2: каталог проєкту після копіювання (source.txt і copy.txt)

Завдання 3. Трекер витрат, який не боїться поганих даних (task3.py)

Трекер витрат із команд **add**, **list**, **report** зроблено стійким до поганих даних. Файл **data/expenses.csv** має рядок-заголовок **date;category;amount;note** і по одній витраті в рядку.

Вихідний файл витрат

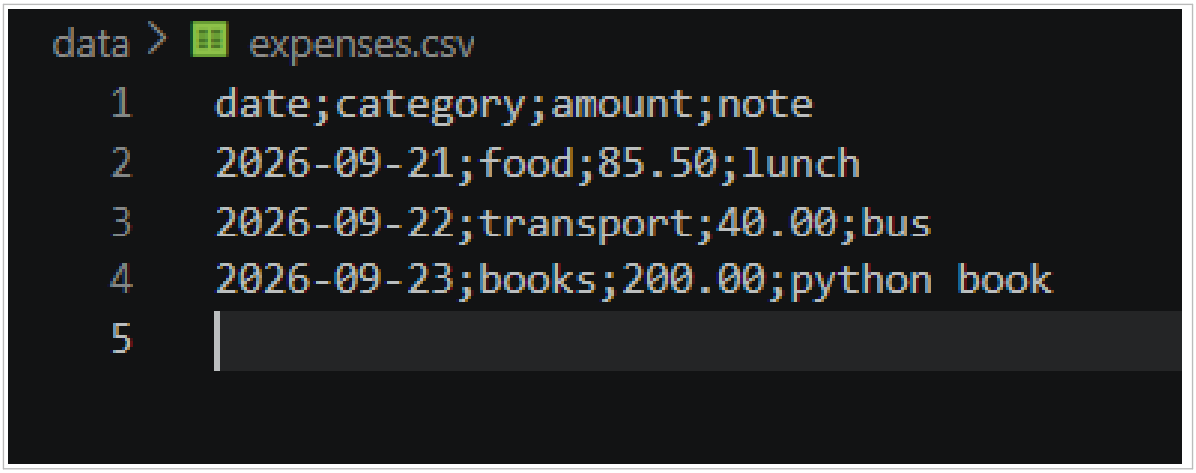


Рис. 10 — Завдання 3: вихідний файл data/expenses.csv (заголовок і три витрати)

Код програми

validate_expense — єдине місце, де перевіряються дані витрати; її використовують і читання файлу, і команда **add**, тому код перевірки не дублюється. Функція перевіряє формат дати (**datetime.strptime**), непорожні категорію й примітку, відсутність символу **;** у полях, що сума є числом (**Decimal**, виняток **InvalidOperation**), скінченна та більша за нуль. За будь-якої проблеми викликається **ValueError** з поясненням.

task3.py

```python

import sys
from pathlib import Path
from datetime import datetime
from decimal import Decimal, InvalidOperation

print("Nikita Krilovets, IT-31")

DATA_FILE = Path("data/expenses.csv")
HEADER = "date;category;amount;note"


def validate_expense(date, category, amount, note):
    try:
        datetime.strptime(date, "%Y-%m-%d")
    except ValueError:
        raise ValueError("неправильна дата, потрібен формат YYYY-MM-DD")

    if not category.strip():
        raise ValueError("категорія не може бути порожньою")

    if not note.strip():
        raise ValueError("примітка не може бути порожньою")

    if any(";" in field for field in (date, category, amount, note)):
        raise ValueError("поля не повинні містити символ ';'")

    try:
        value = Decimal(amount)
    except InvalidOperation:
        raise ValueError("сума має бути числом")

    if not value.is_finite() or value <= 0:
        raise ValueError("сума повинна бути більшою за нуль")

    return value

```

Рис. 11 — Завдання 3: функція validate_expense

read_expenses читає файл у UTF-8 і повертає кортеж: коректні витрати, список пропущених рядків (номер і причина) та помилку файлу. Порожній файл і неправильний заголовок призводять до **ValueError**. Кожен рядок перевіряється: порожній рядок, не 4 поля або помилка з **validate_expense** — рядок пропускається і заноситься до списку пропущених. Проблеми самого файлу (**OSError**, **UnicodeError**, **ValueError**) перехоплюються лише в цій функції — в одному місці програми.

```python

def read_expenses():
    try:
        with DATA_FILE.open("r", encoding="utf-8") as file:
            lines = file.readlines()

        if not lines:
            raise ValueError("файл не містить заголовка")

        if lines[0].strip() != HEADER:
            raise ValueError("неправильний заголовок файлу")

        expenses = []
        skipped = []

        for line_number, line in enumerate(lines[1:], start=2):
            line = line.rstrip("\r\n")

            if not line.strip():
                skipped.append((line_number, "порожній рядок"))
                continue

            fields = line.split(";")

            if len(fields) != 4:
                skipped.append((line_number, "потрібно 4 поля"))
                continue

            date, category, amount, note = fields

            try:
                value = validate_expense(date, category, amount, note)
            except ValueError as error:
                skipped.append((line_number, str(error)))
                continue

            expenses.append({
                "date": date,
                "category": category,
                "amount": value,
                "note": note
            })

        return expenses, skipped, None

    except (OSError, UnicodeError, ValueError) as error:
        return None, [], error

```

Рис. 12 — Завдання 3: функція read_expenses

show_skipped виводить у **sys.stderr** перелік пропущених рядків з номером і причиною та їх загальну кількість. **list_expenses** показує коректні витрати у звичайний вивід, помилку файлу виводить у **sys.stderr** і повертає код 1, а після таблиці викликає **show_skipped**; код завершення — 0.

```python

def show_skipped(skipped):
    for line_number, reason in skipped:
        print(
            f"Пропущено рядок {line_number}: {reason}",
            file=sys.stderr
        )

    if skipped:
        print(
            f"Усього пропущено рядків: {len(skipped)}",
            file=sys.stderr
        )


def list_expenses():
    expenses, skipped, error = read_expenses()

    if error:
        print(
            f"Помилка файлу: {type(error).__name__}: {error}",
            file=sys.stderr
        )
        return 1

    if not expenses:
        print("Немає коректних витрат.")

    for expense in expenses:
        print(
            f'{expense["date"]} | {expense["category"]} | '
            f'{expense["amount"]:.2f} | {expense["note"]}'
        )

    show_skipped(skipped)
    return 0

```

Рис. 13 — Завдання 3: функції show_skipped і list_expenses

report_expenses повідомляє про помилку файлу (код 1). Якщо коректних витрат немає — виводить «Немає коректних витрат для звіту.» і повертає код 1. Інакше виводить кількість витрат, загальну суму (**Decimal**) та суми за категоріями, а пропущені рядки — у **sys.stderr**.

```python

def report_expenses():
    expenses, skipped, error = read_expenses()

    if error:
        print(
            f"Помилка файлу: {type(error).__name__}: {error}",
            file=sys.stderr
        )
        return 1

    if not expenses:
        print("Немає коректних витрат для звіту.")
        show_skipped(skipped)
        return 1

    total = sum(
        (expense["amount"] for expense in expenses),
        Decimal("0")
    )

    print(f"Кількість витрат: {len(expenses)}")
    print(f"Загальна сума: {total:.2f}")

    categories = {}

    for expense in expenses:
        category = expense["category"]
        categories[category] = (
            categories.get(category, Decimal("0")) + expense["amount"]
        )

    print("Суми за категоріями:")

    for category, amount in categories.items():
        print(f"{category}: {amount:.2f}")

    show_skipped(skipped)
    return 0

```

Рис. 14 — Завдання 3: функція report_expenses

add_expense перевіряє кількість аргументів (інакше підказка і код 2), перевіряє дані через **validate_expense** (при помилці — повідомлення, код 1 і файл не змінюється), перевіряє, чи файл придатний для читання, і лише потім дописує рядок у режимі **"a"**. Помилки запису перехоплюються й дають код 1.

```python

def add_expense(args):
    if len(args) != 4:
        print(
            "Використання: python task3.py add <дата> <категорія> "
            "<сума> <примітка>",
            file=sys.stderr
        )
        return 2

    date, category, amount, note = args

    try:
        value = validate_expense(date, category, amount, note)
    except ValueError as error:
        print(f"Некоректні дані: {error}", file=sys.stderr)
        return 1

    expenses, _, error = read_expenses()

    if error:
        print(
            f"Помилка файлу: {type(error).__name__}: {error}",
            file=sys.stderr
        )
        return 1

    try:
        with DATA_FILE.open("a", encoding="utf-8") as file:
            file.write(
                f"{date};{category};{value:.2f};{note}\n"
            )
    except (OSError, UnicodeError) as error:
        print(
            f"Не вдалося записати файл: {type(error).__name__}: {error}",
            file=sys.stderr
        )
        return 1

    print("Витрату додано.")
    return 0

```

Рис. 15 — Завдання 3: функція add_expense

Функція **main()** розбирає команду: без команди — підказка і код 2; **add**, **list**, **report** викликають відповідні функції (для **list** і **report** зайві аргументи дають підказку і код 2); невідома команда — повідомлення в **sys.stderr** і код 2.

```python

def main():
    if len(sys.argv) < 2:
        print(
            "Використання: python task3.py add/list/report",
            file=sys.stderr
        )
        return 2

    command = sys.argv[1]

    if command == "add":
        return add_expense(sys.argv[2:])
    elif command == "list":
        if len(sys.argv) != 2:
            print("Використання: python task3.py list", file=sys.stderr)
            return 2
        return list_expenses()
    elif command == "report":
        if len(sys.argv) != 2:
            print("Використання: python task3.py report", file=sys.stderr)
            return 2
        return report_expenses()
    else:
        print(f"Невідома команда: {command}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    sys.exit(main())

```

Рис. 16 — Завдання 3: функція main()

Результати перевірки

На одному знімку показано всю послідовність перевірок:

- Зроблено копію файлу (**expenses-good.csv**) і додано до **expenses.csv** чотири зіпсовані рядки: сума текстом (*sixty five*), відсутнє поле, неіснуюча дата *2026-13-14* та від'ємна сума *-32.00*.
- **list** — показано три коректні витрати, а в **sys.stderr** — пропущені рядки 5–8 з причинами та їх кількість (4); код 0.
- **add 2026-09-28 food ninety pizza** — «Некоректні дані: сума має бути числом», нічого не записано; код 1.
- Файл замінено каталогом з назвою **data/expenses.csv** — **list** виводить «Помилка файлу: PermissionError», код 1.
- Файл замінено лише заголовком — **report** виводить «Немає коректних витрат для звіту.», код 1.
- **summary** — невідома команда; код 2.

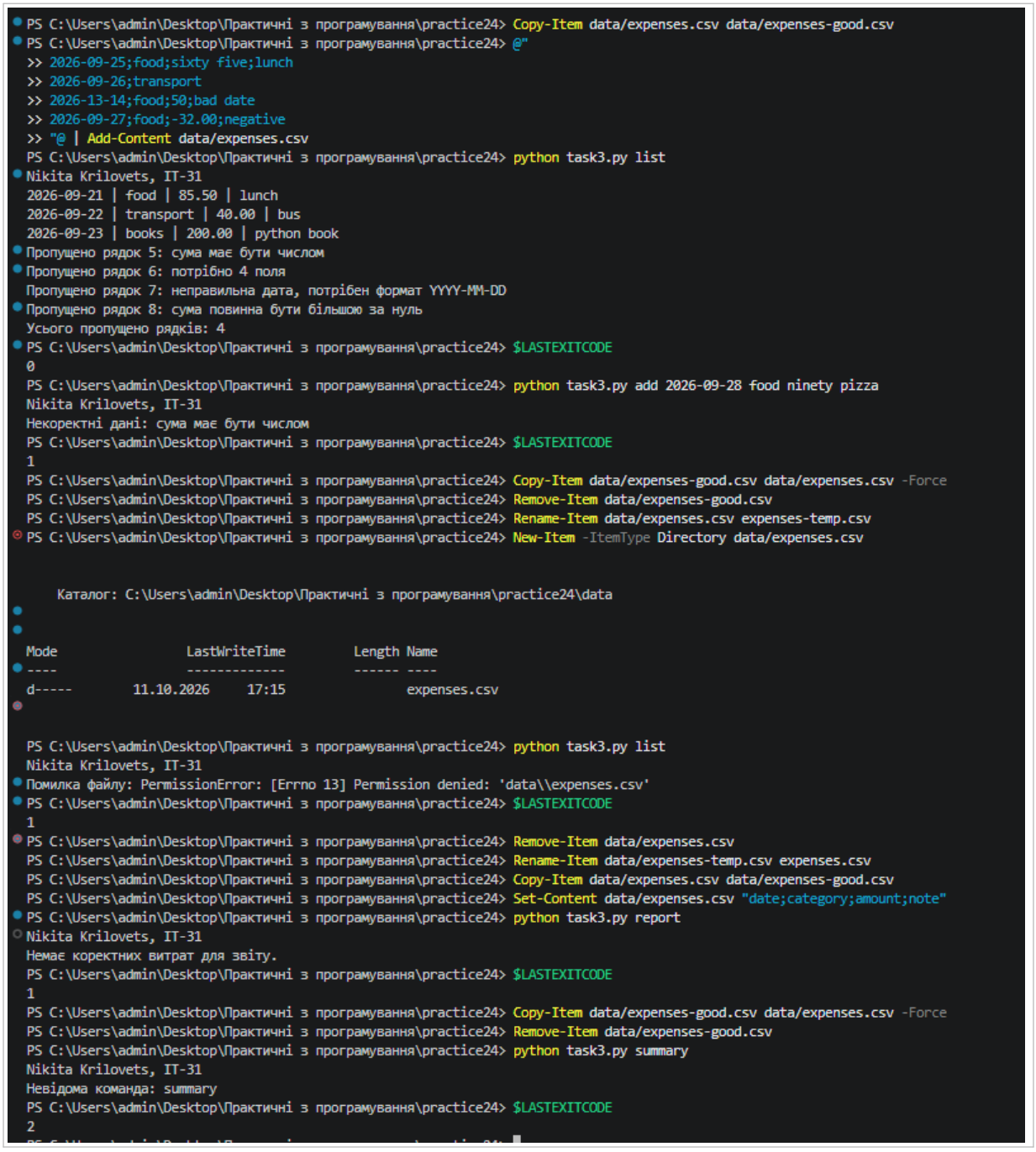


Рис. 17 — Завдання 3: перевірка програми на поганих даних та коди завершення

Відповідь на питання до звіту

Питання: чому після **python task3.py list > list.txt** перелік пропущених рядків з'являється на екрані, а не у файлі?

Відповідь: оператор **>** перенаправляє у файл лише стандартний потік виводу (**stdout**, дескриптор 1). Таблиця витрат виводиться через **print()** саме в **stdout**, тому потрапляє у **list.txt**. Перелік пропущених рядків програма виводить у стандартний потік помилок (**stderr**, дескриптор 2) — **print(..., file=sys.stderr)**. Цей потік окремий і не перенаправляється оператором **>**, тому залишається на екрані. Щоб зберегти і помилки, потрібно перенаправити потік 2 окремо, наприклад **2> errors.txt**, або обидва потоки разом: **> list.txt 2>&1**.